Software Design Document (SDD)

Cover

Field	Value
Project	Department Course Timetable Management System
Document	Software Design Document
Version	v2
Related SRS version	v2
Date	10.11.2026
Team number	Team 21
Member 1	[Buğrahan KOÇASLAN] · [admbugraa]
Member 2	[Furkan KÖRPE] · [Furkankrp]
Repository	https://github.com/Furknkrp/-Timetable-Management-System

Revision History

Version	Date	Author(s)	Summary of changes
v1	01.11.2026	Buğrahan KOÇASLAN, Furkan KÖRPE	Initial design specification containing structural class diagram and algorithm flowchart.
v2	10.11.2026	Buğrahan KOÇASLAN, Furkan KÖRPE	Complete, unified SDD update for Code v1. Synchronized class structure with active Screen 1 console menu implementation and established class stubs, full constraint checks, traceability matrix, and conflict resolution rules for full build integration.

---

1. Introduction

This Software Design Document (SDD) provides a comprehensive structural and behavioral design specification for the Department Course Timetable Management System. It presents the complete architectural blueprint, class structures, memory models, constraint verification logic, and user interface workflows.

This document aligns with **SRS v2** and corresponds to the **Code v1** release milestone. In Code v1, the complete object-oriented class hierarchy, data entities, and algorithm workflows are established, while system execution specifically focuses on rendering and operating **Screen 1 (Main Menu Navigation)** to drive application workflow.

2. System Overview

The application is structured into five core functional modules. In the Code v1 implementation, **Screen 1** provides the console interface connecting all modules, with underlying domain entities and scheduler methods defined to support upcoming functional releases.

Part	Classes	Responsibility	Implementation Status (Code v1)
User Interface	Main	Renders Screen 1 (Main Menu), captures user keyboard input via Scanner, handles navigation, and routes menu choices (0–8).	**Active / Fully Functional** (Renders Screen 1 and handles options 0–8).
Input Parsing	DataLoader	Reads and parses courses.csv, instructors.txt, rooms.csv, and constraints.txt; handles line errors and logs warnings.	**Structured Stub** (Ready for file parsing in Code v2).
Domain Model	Course, Session, Instructor, Room, TimeSlot, SpecialRequest	Models core academic entities, time slot intervals, room capacities, room types, and session assignment states.	**Fully Defined Entities** (Classes, attributes, getters/setters ready).
Scheduling Core	Scheduler	Implements Phase 1 & 2 algorithms, unified Hard/Soft	**Core Logic Mapped / Method Stubs Established**.
Part	**Classes**	**Responsibility**	**Implementation Status (Code v1)**
		constraint verification, manual modification validation, and email drafting.	
Output / State	Timetable	Stores collections of scheduled sessions, unscheduled sessions, and warnings; prepares console grid outputs and disk files.	**Data Structure Ready** (Holders for sessions and warnings ready).

3. UML Class Diagram

The UML Class Diagram defines the complete object-oriented structure of the system. All classes, attributes, methods, and relationships shown below match the Java codebase architecture introduced in Code v1.

```
None
classDiagram
    class Main {
        +main(String[] args) void
        +displayMenu() void
        +handleUserChoice(int choice) void
    }
    class DataLoader {
        +loadCourses(String path) List~Course~
        +loadInstructors(String path) List~Instructor~
        +loadRooms(String path) List~Room~
        +loadRequests(String path) List~SpecialRequest~
    }
    class Course {
        - String courseCode
        - String title
        - int yearGroup
        - int theoryHours
        - int labHours
        - int studentCount
        - Instructor instructor
        +getCourseCode() String
        +getYearGroup() int
        +getStudentCount() int
        +getInstructor() Instructor
    }
```

```
class Session {
    -String sessionId
    -Course course
    -String sessionType
    -int durationHours
    -String requiredRoomType
    -TimeSlot assignedSlot
    -Room assignedRoom
    -boolean scheduled
    -String failureReason
    -String problemType
    +assign(TimeSlot slot, Room room) void
    +unassign() void
    +isScheduled() boolean
    +getSessionId() String
}
class Instructor {
    -String instructorId
    -String name
    -List~TimeSlot~ unavailableSlots
    -List~TimeSlot~ preferredSlots
    +isAvailable(TimeSlot slot) boolean
    +prefers(TimeSlot slot) boolean
}
class Room {
    -String roomId
    -String roomType
    -int capacity
    -List~TimeSlot~ unavailableSlots
    +isAvailable(TimeSlot slot) boolean
    +canHost(Session session) boolean
}
class TimeSlot {
    -String day
    -int startHour
    -int endHour
    +overlapsWith(TimeSlot other) boolean
    +toString() String
}
class SpecialRequest {
    -String courseCode
    -String requestType
    -boolean mandatory
    -TimeSlot fixedSlot
    -String requiredRoomId
    +isMandatory() boolean
    +getFixedSlot() TimeSlot
}
```

```
class Timetable {
  -List~Session~ scheduledSessions
```

```
    -List~Session~ unscheduledSessions
    -List~String~ warnings
    +addScheduled(Session session) void
    +addUnscheduled(Session session, String problemType, String reason) void
    +addWarning(String warning) void
    +printYearGroupTimetable(int year) void
    +printUnscheduledAndWarnings() void
    +saveToFile(String filePath) void
  }
  class Scheduler {
    -List~Course~ courses
    -List~Instructor~ instructors
    -List~Room~ rooms
    -List~SpecialRequest~ requests
    -Timetable timetable
    +loadAllData() void
    +generateTimetable() void
    +checkHardConstraints(Session s, TimeSlot t, Room r) String
    +checkSoftConstraints(Session s, TimeSlot t) List~String~
    +modifyAssignment(String sessionId, TimeSlot newSlot, Room newRoom)
boolean
    +generateClassroomRequestEmail(Session s, String outputPath) void
  }

  Main --> Scheduler : uses
  Scheduler --> DataLoader : relies on
  Scheduler --> Timetable : manages
  Scheduler --> Course : operates on
  Scheduler --> Instructor : operates on
  Scheduler --> Room : operates on
  Scheduler --> SpecialRequest : evaluates
  Course --> Instructor : taught by
  Session --> Course : derived from
  Session --> TimeSlot : assigned to
  Session --> Room : placed in
  Timetable --> Session : aggregates
```

Class Diagram

Verification Checklist:

- [x] Every main class is shown, named exactly as in the Java code (Main, Scheduler, Course, Session, Instructor, Room, TimeSlot, SpecialRequest, Timetable, DataLoader).
- [x] Important attributes have a type and visibility (e.g., - capacity: int).
- [x] Main methods have parameters and a return type (e.g., + isAvailable(slot: TimeSlot): boolean).
- [x] Relationships between classes are drawn with appropriate multiplicities.
- [x] The class that runs the scheduling algorithm (Scheduler) is clearly identifiable. ● [x] Diagram layout is legible and structured for single-page presentation.

---

4. Class Descriptions

	Class	Responsibility	Key Attributes	Key Methods	Related Classes
	Main	Drives application life cycle, renders Screen 1 (Main Menu), reads command-line inputs via Scanner, validates choices, and delegates actions to Scheduler.	scanner: Scanner, scheduler: Scheduler	main(args), displayMenu(), handleUserChoice(c)	Scheduler
	DataLoader	Handles external file I/O operations for .csv and .txt files in data/, parsing data into domain entities and capturing format errors.	warningLogs: List<String>	loadCourses(path), loadRooms(path), loadInstructors(path)	Course, Room, Instructor
	Course	Encapsulates curriculum information (course code, title, year group, theory/lab duration, student count, assigned instructor).	courseCode, yearGroup, theoryHours, labHours, studentCount	getCourseCode(), getYearGroup(), getStudentCount()	Instructor, Session
	Session	Represents an atomic teaching unit (-T for theory or -L for lab) derived from a course. Tracks slot, room assignment, and failure state.	sessionId, sessionType, durationHours, assignedSlot, assignedRoom	assign(slot, room), unassign(), isScheduled()	Course, TimeSlot, Room
	Instructor	Stores instructor metadata, assigned courses, and unavailable/preferred scheduling time slots.	instructorId, name, unavailableSlots, preferredSlots	isAvailable(slot), prefers(slot)	TimeSlot, Course
	Room	Models physical classrooms and laboratories, keeping track of room capacity, room type, and reserved unavailable slots.	roomId, roomType, capacity, unavailableSlots	canHost(session), isAvailable(slot)	Session, TimeSlot

Class	Responsibility	Key Attributes	Key Methods	Related Classes
TimeSlot	Defines a day-and-hour temporal block (e.g., Monday 09:00–12:00) and executes overlap calculations across schedule intervals.	day, startHour, endHour	overlapsWith(other), toString()	Session
SpecialRequest	Stores constraint rules from constraints.txt (fixed slots, specific room requirements, mandatory flags).	courseCode, requestType, mandatory, fixedSlot, requiredRoomId	isMandatory(), getFixedSlot()	Course, TimeSlot
Timetable	Holds in-memory collections of scheduled sessions, unscheduled sessions, and soft warnings. Manages display grids and output file saving.	scheduledSessions, unscheduledSessions, warnings	addScheduled(s), printYearGroupTimetable(year), saveToFile(path)	Session
Scheduler	Manages automatic scheduling logic, executes Hard/Soft constraint checks, handles manual modifications, and drafts emails.	courses, rooms, instructors, requests, timetable	generateTimetable(), checkHardConstraints(s,t ,r), modifyAssignment(...)	Timetable, DataLoader

5. Design Decisions

Question	**Our Decision**	**Why**
How is a course split into sessions?	Each Course object instantiates one Theory Session (<Code>-T) if theoryHours > 0 and one Lab Session (<Code>-L) if labHours > 0.	Keeps theory and lab requirements separate (e.g., different room types and durations) while enforcing that each block is scheduled as an unbroken unit.
How is a time slot represented?	Modeled by TimeSlot containing a String day ("Mon"–"Fri"), int startHour (9–16), and int endHour (10–17).	Provides precise mathematical overlap detection (start1 < end2 && start2 < end1) for contiguous multi-hour session blocks.

Question	**Our Decision**	**Why**
How is the timetable stored?	Maintained within Timetable using flat ArrayList<Session> collections, filtered dynamically on demand by Year Group, Room, or Instructor.	Avoids redundant multi-dimensional data structures and simplifies updating slot assignments during manual modifications.
How are instructor and room availability stored?	Stored as List<TimeSlot> attribute lists inside Instructor and Room classes.	Allows fast lookup using isAvailable(targetSlot) which iterates over configured unavailable slots.
How are special requests represented?	Parsed into SpecialRequest objects containing request type enums (FIXED_TIME, SPECIFIC_ROOM) and a boolean mandatory flag.	Enables the scheduler to treat mandatory requests as hard constraints (Phase 1) and preferred requests as soft constraint heuristics.
How are hard constraints checked?	Consolidated into a single method: Scheduler.checkHardConstraints(Session s, TimeSlot t, Room r).	Guarantees 100% consistency between automatic algorithm decisions and interactive manual modifications, returning null if valid or a violation message.

How are input errors detected and reported?	Handled in DataLoader using try-catch blocks around parsing loops. Malformed lines log a warning message to Timetable.warnings and processing continues.	Satisfies NFR-01 and FR-04 by preventing system crashes due to minor syntax issues in input data files.
Which collections are used, and for what?	ArrayList for ordered lists of sessions, rooms, instructors, and warnings; HashMap<String, Course> for fast course code lookups.	ArrayList provides fast traversal during algorithm loops; HashMap provides key-based lookup when$O(1)$ resolving course references.

---

6. Automatic Scheduling Algorithm

6.1 Strategy

1. **Processing Order:** Phase 1 processes sessions linked to mandatory FIXED_TIME special requests. Phase 2 processes remaining sessions, prioritizing Theory (-T) before Lab (-L) sessions, ordered descending by expected student count.
2. **Candidate Time Slot Selection:** For a session of duration , the algorithm iterates

   through working days (Monday to Friday) and valid candidate$D$ start hours

   .
3. **Suitable**$H \in [9, 17$ **Room**$- D]$ **Selection:** Iterates through available rooms, filtering strictly by

Room.roomType == Session.requiredRoomType and Room.capacity >= Session.studentCount.

4. **Constraint Verification:** Evaluates candidate (Session, TimeSlot, Room) against checkHardConstraints().
5. **Conflict Handling:** If a hard constraint fails, the algorithm advances to the next candidate room, then to the next candidate time slot.
6. **Unscheduled Decision:** If all 40 time slots and all suitable rooms are exhausted without finding a valid combination, the session is marked as UNSCHEDULED, and the reason (NO ROOM or CONSTRAINT) is recorded.

6.2 Flowchart

```
None flowchart
TD
  A([Start Timetable Generation]) --> B[DataLoader: Read courses, instructors,
```

```
rooms, requests]
        B  --> C[Construct Session Objects: -T and -L blocks]
        C  --> D[Phase 1: Filter Mandatory Fixed-Time Requests]
        D  --> E{More Fixed-Time Sessions?}
        E  -- Yes --> F[Select Fixed Session & Specified Slot/Room]
        F  --> G{checkHardConstraints HC-01..HC-09 == null?}
        G  -- Yes --> H[Assign Session to Slot & Room -> Scheduled]
        G  -- No --> I[Mark Session UNSCHEDULED -> Reason: CONSTRAINT]
        H  --> E
    I --> E
    E -- No --> J{Phase 2: More Unscheduled Sessions?}
        J  -- Yes --> K[Select Next Session: Theory first, then Lab]
        K  --> L[Iterate Days Mon-Fri & Valid Start Hours 09:00 to 17-Duration]
        L  --> M{Valid TimeSlot Candidate?}
        M -- Yes --> N[Iterate Rooms matching Type & Capacity]
        N  --> O{Candidate Room Available?}
        O  -- Yes --> P{checkHardConstraints HC-01..HC-09 == null?}
        P  -- Violated --> N
          P  -- Satisfied --> Q[Assign Session to Slot and Room]
          Q  --> R{checkSoftConstraints SC-01..SC-03}     R -- Violated --> S[Log Soft-Constraint Warning Message]
          R  -- Satisfied --> J
          S  --> J
            O -- No Rooms Left --> L
            M -- No Slots Left --> T[Mark Session UNSCHEDULED: Record NO ROOM or
          CONSTRAINT]
            T --> J
            J -- No --> U[Format Timetable Grids & Summary Reports]
            U --> V([End Process])
```

6.3 Where Each Hard Constraint Is Checked

ID	Constraint	Checked in (Class.method)	Code Version
HC-01	Instructor conflict	Scheduler.checkHardConstraints()	Planned for Code v3
HC-02	Room conflict	Scheduler.checkHardConstraints()	Planned for Code v3
HC-03	Year-group conflict	Scheduler.checkHardConstraints()	Planned for Code v3
HC-04	Working hours	TimeSlot constructor & Scheduler.checkHardConstraints()	Planned for Code v3
HC-05	Room capacity	Room.canHost() & Scheduler.checkHardConstraints()	Planned for Code v3
HC-06	Room type	Room.canHost() & Scheduler.checkHardConstraints()	Planned for Code v3

HC-07	Instructor availability	Instructor.isAvailable() & Scheduler.checkHardConstraints()	Planned for Code v3
HC-08	Fixed-time requirements	Scheduler.generateTimetable() (Phase 1)	Planned for Code v3
ID	**Constraint**	**Checked in (Class.method)**	**Code Version**
HC-09	Room availability	Room.isAvailable() & Scheduler.checkHardConstraints()	Planned for Code v3

6.4 Known Limitations

- **First-Fit Greedy Myopia:** The First-Fit heuristic does not perform backtracking. Placing a large course in an early slot may prevent multiple smaller courses from being placed later, leading to sub-optimal space utilization.
- **Lack of Dynamic Swap Chains:** If a session remains unscheduled due to a minor conflict, the algorithm cannot automatically swap previously scheduled flexible courses to make room.

---

7. Conflict Detection and Handling

Problem	Detected in (Class.method)	What the Algorithm Does	What the User Sees
Instructor or room busy	Scheduler.checkHardConstraints()	Skips current slot/room and tests the next candidate room or time slot.	No error during generation; final grid shows non-overlapping assignments.
Year-group overlap	Scheduler.checkHardConstraints()	Skips slot where another course of the same Year Group is already scheduled.	Ensures student cohort has no concurrent classes.
No room with capacity/type	Scheduler.checkHardConstraints()	Marks session as UNSCHEDULED, sets problemType = "NO ROOM".	Listed in Screen 5 under Unscheduled Sessions; enables Screen 7 email option.
Fixed-time constraint violation	Scheduler.generateTimetable()	Rejects fixed assignment, marks session as UNSCHEDULED, sets problemType = "CONSTRAINT".	Logged in Screen 3 execution summary and Screen 5 unscheduled report.
Soft-constraint violation	Scheduler.checkSoftConstraints()	Completes assignment, generates a warning string, and appends it to Timetable.warnings.	Screen 5 displays warning message (e.g., theory scheduled after lab).

---

8. User Interface Design & Screen 1 Implementation

In **Code v1**, **Screen 1 (Main Menu)** is active and fully operational in console execution. The user can select menu options 0 to 8, which display informative status placeholders for unimplemented functions while keeping the execution loop active until option 0 (Exit) is selected.

None

==========

```
DEPARTMENT COURSE TIMETABLE MANAGEMENT SYSTEM
==================================================
1. Load input data
2. Display data load status & warnings
3. Run automatic timetable generation
4. Display weekly timetable by year group
5. Display unscheduled sessions & warnings
6. Modify/remove session assignment
7. Generate classroom request email
8. Export timetable to file
0. Exit ================================================== Enter
choice (0-8):
```

	Menu Option	What It Does	Requirement IDs	Class.method Called	Code v1 Implementation Status
	1. Load input data	Triggers loading and parsing of all data files from data/.	FR-01, FR-02, FR-03, FR-04	Scheduler.loadAllData()	**Active Screen 1 Menu Route** (Stub response in v1).
	2. Display data load status & warnings	Prints counts of loaded courses, rooms, instructors, and parsing warnings.	FR-04	Timetable.printLoadStatu s()	**Active Screen 1 Menu Route** (Stub response in v1).
	3. Run automatic timetable generation	Executes Phase 1 and Phase 2 scheduling algorithms.	FR-05, FR-06	Scheduler.generateTimeta ble()	**Active Screen 1 Menu Route** (Stub response in v1).
	4. Display weekly timetable by year group	Renders 5-day grid schedule for selected Year Group (1–4).	FR-09	Timetable.printYearGroup Timetable()	**Active Screen 1 Menu Route** (Stub response in v1).
	5. Display unscheduled sessions & warnings	Lists unplaced courses with failure reasons and soft warnings.	FR-07, FR-08	Timetable.printUnschedul edAndWarnings()	**Active Screen 1 Menu Route** (Stub response in v1).

	Menu Option	What It Does	Requirement IDs	Class.method Called	Code v1 Implementation Status
	6. Modify/remove session assignment	Allows manual reassignment or removal of a session with validation checks.	FR-10	Scheduler.modifyAssignme nt()	**Active Screen 1 Menu Route** (Stub response in v1).
	7. Generate classroom request email	Drafts official room request email (.txt) for unplaced sessions.	FR-11	Scheduler.generateClassr oomRequestEmail()	**Active Screen 1 Menu Route** (Stub response in v1).
	8. Export timetable to file	Saves complete timetable reports to output/ folder.	FR-12	Timetable.saveToFile()	**Active Screen 1 Menu Route** (Stub response in v1).
	0. Exit	Terminates the console application gracefully.	NFR-02	System.exit(0)	**Fully Implemented** (Exits application loop cleanly).

---

9. Requirements Traceability

Requirement	Implemented in (Class.method)	Test Case(s)	Code v1 Status
FR-01	DataLoader.loadCourses()	TC-12	Method Stub Defined
FR-02	DataLoader.loadInstructo rs()	TC-06, TC-12	Method Stub Defined
FR-03	DataLoader.loadRooms()	TC-05, TC-12	Method Stub Defined
FR-04	DataLoader error handling logic	TC-12	Logic Designed
FR-05	Scheduler.generateTimeta ble()	TC-07	Algorithm Mapped

FR-06	Scheduler.generateTimeta ble()	TC-01, TC-09	Algorithm Mapped
FR-07	Timetable.addUnscheduled ()	TC-10	Data Structure Defined
FR-08	Scheduler.checkSoftConst raints()	TC-08, TC-14	Logic Designed

Requirement	**Implemented in (Class.method)**	**Test Case(s)**	**Code v1 Status**
FR-09	Timetable.printYearGroup Timetable()	TC-01	UI Grid Format Designed
FR-10	Scheduler.modifyAssignme nt()	TC-13	Validation Rules Mapped
FR-11	Scheduler.generateClassr oomRequestEmail()	TC-11	Output Template Defined
FR-12	Timetable.saveToFile()	TC-01	File Path Configured
HC-01	Scheduler.checkHardConst raints()	TC-02	Specification Ready
HC-02	Scheduler.checkHardConst raints()	TC-03	Specification Ready
HC-03	Scheduler.checkHardConst raints()	TC-04	Specification Ready
HC-04	TimeSlot & Scheduler.checkHardConst raints()	TC-01	Class Defined
HC-05	Room.canHost() & Scheduler.checkHardConst raints()	TC-05	Specification Ready
HC-06	Room.canHost() & Scheduler.checkHardConst raints()	TC-05	Specification Ready
HC-07	Instructor.isAvailable() & Scheduler.checkHardConst raints()	TC-06	Specification Ready
HC-08	Scheduler.generateTimeta ble()	TC-07	Phase 1 Mapped
Requirement	**Implemented in (Class.method)**	**Test Case(s)**	**Code v1 Status**

HC-09	Room.isAvailable() & Scheduler.checkHardConstraints()	TC-03	Specification Ready

---

10. Document Revisions

Element	Change	Reason
System Overview (Section 2)	Consolidated software component definitions and clarified Code v1 operational status.	Establishes single-document clarity for evaluation checkpoints.
UML & Class Descriptions (Section 3 & 4)	Full structural definitions provided for all 10 project classes.	Ensures 1:1 mapping with Java source code structure.
UI Design & Screen 1 (Section 8)	Detailed interactive menu execution for Screen 1.	Accurately reflects Code v1 runtime capability.

AI-Use Declaration

Tool	Used for	How We Checked the Result
Gemini	Generating Mermaid class diagram syntax and formatting template structures.	Inspected rendered Mermaid diagrams on mermaid.live and verified class names against Java source files in src/.
Gemini	**Structuring SDD Sections & Content Formatting:** Assisting in organizing document sections according to project template guidelines, generating Mermaid syntax for class and flowchart diagrams, and formatting	Both team members thoroughly reviewed all generated section structures to match SENG 271 course specifications.